

PaperTrail | Web-first roadmap

Revision 2 - 1 October 2026

A browser-first, local-first PDF and EPUB reader built with Vue 3, Vite and TypeScript. The first release runs in a browser. Tauri and native distribution are a later product phase.

Repository: HimanshuHD/papertrail-reader (private)

Decision: #30. Parent roadmap: #1. Bootstrap PR: #29 (open at revision time).

User journey: choose a folder or files explicitly; discover supported documents; browse the directory sidebar; read one file in the main viewer; save positions and bookmarks locally.

Release 0.1: M0-M4 plus saved positions/bookmarks #13, release-relevant reliability checks, web build and tested browser delivery. Advanced annotations, tabs and split view follow.

Source visibility and deployment audience are separate decisions. Hosting/audience will be agreed before publication. No document upload is required for reading.

This roadmap supersedes the desktop-first plan. Completed repository work remains completed; deferred desktop work remains open. The issues and progress tracker hold current execution status.

1. Browser product scope

Folder selection must follow user interaction. Feature-detect `showDirectoryPicker`; it requires a secure context and is not universally available. Support directory file input with `webkitdirectory` and relative paths; fall back to selecting individual PDF/EPUB files.

Browser selection is a permission boundary, not unrestricted hard-drive access. Directory inputs return snapshots of files; the app cannot promise continuous filesystem watching. Refresh a live handle when available or ask for reselection. The browser/OS picker may enumerate before files reach the app; cancellation applies to app indexing after selection.

Saved metadata does not guarantee future file access. Reuse persisted handles only where supported and after checking permissions; otherwise restore reading state once the user reselects matching files. Do not store full documents by default.

P0: folder/file selection, incremental indexing, tree, single reader, loading/error/empty states, PDF page navigation/progress/zoom/fit/scroll, EPUB chapters/contents/font controls.

P1: positions, bookmarks, search, text selection, outlines, fullscreen, keyboard controls and themes.

P2/P3: annotations, statistics, tabs, session restoration, split view, annotation export and more formats.

PDF has fixed pages. EPUB uses chapters, CFI/location and percentage; reflow changes any displayed pagination. OCR for image-only PDFs and DRM-protected EPUB support are outside 0.1.

2. Architecture and persistence

UI: Vue 3 + TypeScript. Build: Vite. Styling/state/routing: Tailwind, Pinia and Vue Router as their issues are delivered.

Components call application services and Pinia actions. A LibraryProvider abstracts file selection, indexing, read access, capability checks and refresh. BrowserLibraryProvider uses File/Blob objects and optional directory handles. A future TauriLibraryProvider will implement native access behind the same boundary.

PDF and EPUB readers remain separate adapters behind a shared reader contract: open, close, navigation, progress, contents and supported controls. PDF.js uses a worker and lazy visible-page rendering. epub.js handles reflow and CFI navigation.

Versioned IndexedDB stores document identity, reading positions, bookmarks and preferences. Use generated IDs and lazy fingerprints to reconcile reselected/moved files; handle ambiguous matches explicitly. Avoid whole-library eager hashing. Handle unavailable/cleared storage and quota errors.

Revoke Blob URLs and dispose readers on switching. Bound EPUB archive processing; disable book scripting and restrict remote resources. Validate malformed/password PDFs and unsupported EPUB content.

Folders: src/components/{layout,library,viewer}; src/features/{library,pdf,epub,bookmarks}; src/{composables,router,stores,types,views,assets}; tests/{unit,e2e}; docs/; .github/. Add src-tauri only in the later desktop phase.

3. Milestones M0-M2

M0 - Engineering foundation (#2, #3, #5; children #21, #22)

Repository docs #2 are completed. Frontend bootstrap #3 is in review in PR #29. Establish committed lockfile, strict types and production build. Expand quality checks/tests #22. Merge tracking #21 requires a real-merge verification. Exit: clean CI install/type/build; tooling evidence recorded. Tauri #4 and Rust #23 are deferred.

M1 - UI and state (#6, #7)

Add tokens, themes, Pinia/Router, format capability contracts, accessible split layout, sidebar, toolbar and status states. Exit: keyboard access, focus visibility and small-window usability verified; sample content clearly identified.

M2 - Browser library (#8, #9; children #24, #25)

Add user-triggered folder selection, feature detection, directory-input and individual-file fallbacks. Index selected files incrementally, reconstruct hierarchy from relative paths, filter case-insensitive extensions and support cancellation. Explain handle refresh versus snapshot reselection.

Exit: nested files, empty selections, permission denial, picker dismissal, unsupported APIs, Unicode paths and large selected libraries handled. Browser selection boundaries are respected. Individual-file fallback never invents directory paths.

Dependencies: #8 follows #3/#6; #24 follows #8; #25 follows #24/#7. #9 is complete only when both child scopes pass acceptance.

4. Milestones M3-M5

M3 - PDF reader (#10, #11)

Integrate PDF.js with worker assets, lazy pages and text layer. Add current/total pages, page jump/slider, continuous scrolling, zoom and fit modes. Add outlines, text search, selection and shortcuts.

Exit: navigation stays synchronized; obsolete render tasks cancel; password/corrupt files recover; text search limitation for scanned pages is explicit; resources released.

M4 - EPUB reader (#12)

Integrate epub.js behind the reader contract. Add chapters/contents, fonts, reading themes and CFI progress. Restrict scripts/remote resources and handle malformed archives.

Exit: reading position survives font/window changes; location generation does not freeze UI; contents navigation works.

M5 - Productivity (#13, #14, #15)

For 0.1 ship IndexedDB reading positions/bookmarks #13. Match reselected files before restoration; handle cleared storage and unavailable permissions. Follow with recents/library search #14 and annotations/statistics #15.

Exit: bookmarks and positions restore after reopening/reselection; migrations and ambiguous identity are tested. Library search is distinguished from document search. Annotation selectors and migration behavior are tested before notes ship.

Dependencies: #10 follows #7/#9; #11 follows #10; #12 follows #7/#9; #13 follows #9/#10/#12. M5 expansion must not silently become a first-release blocker.

5. Milestones M6-M7 and desktop later

M6 - Browser reliability (#16)

Measure scan/index throughput, first-page latency, rendering and memory against a named device/browser and fixture set. Verify resource cleanup, large PDFs/EPUBs, corrupt files, archive bounds, cancellation and restore failures.

Exit: recorded budgets; responsive indexing; no sustained memory growth when switching; recovery paths verified. Release-relevant checks apply to 0.1 even though full performance expansion continues.

M7 - Web release and expansion (#17, #18, #19)

Web release #19: reviewed static production build/deployment #26 and supported-browser validation #28.

Agree hosting and deployment audience before publishing. Verify HTTPS, worker URLs, base paths and release notes.

Tabs/session restoration #17 follow the first release, with isolated viewer state and inactive-resource limits.

Split view/export #18 follow annotations and tabs.

Desktop phase - later (#4; children #23, #27)

Reuse the web UI and readers with TauriLibraryProvider. Add Rust filesystem commands/capabilities, native compile/testing #23 and installer/signing/notarization #27. Create concrete packaging/platform tasks when desktop scope is approved. These issues remain deferred, not completed, and do not block web 0.1.

No current desktop installer or signing claim is made. Browser compatibility is declared from actual tested versions and selection fallbacks, not from an assumption that all APIs exist everywhere.

6. GitHub workflow and issue tracking

GitHub-first development continues: issue -> branch from main -> issue-linked commits -> PR -> CI -> review -> merge. No setup required on the owner's computer.

Frontend CI: clean npm ci from the committed lockfile, strict Vue/config type checks and production build. #22 adds lint, formatting and meaningful unit/component tests. Browser E2E covers selection fixtures, navigation, persistence and fallbacks. Rust CI waits for the desktop phase.

Post-merge #21: reconcile completed issue evidence, parent checklists and docs/progress.md/roadmap.md. Real merge verification is pending; do not declare this automation complete until it writes successfully. Keep partial and deferred issues open.

Status lifecycle: Backlog -> In progress -> In review -> Completed. Also use Blocked or Deferred with explicit reasons. PRs use Closes only for completed scope and Refs for partial work. Completed history is retained.

Issue parent mapping: #5 -> #21/#22; #9 -> #24/#25; #19 -> #26/#28; deferred #4 -> #23/#27. Current connector uses reciprocal links/task lists rather than native sub-issue relations.

Commit lockfiles, pin action SHAs and restrict token permissions. Release publication must respect agreed audience. Private GitHub source is not proof of private hosting.

Next after PR #29: finish frontend quality #22 and UI/state #6, then browser selection #8. Tauri is no longer the next implementation step.

7. Release acceptance and sources

Web 0.1 checklist

- Explicit folder or file selection yields the correct supported-document list/tree.
- Folder capability fallback and permission/dismissal states work.
- PDF navigation/progress/zoom and EPUB chapters/CFI/font controls work.
- Search/contents and basic accessibility are verified.
- Positions/bookmarks restore after permission checks or file reselection.
- Corrupt, password, missing and unsupported files have recoverable states.
- Document contents are processed client-side without upload.
- Storage clearing, quota errors and resource cleanup are tested.
- Clean install/type/build and appropriate lint/test/E2E checks pass.
- Tested browser versions, limitations, HTTPS delivery and deployment audience are documented.

Browser targets: current Chrome/Edge, Firefox and Safari at release time. Test feature detection and fallbacks on each; record exact versions. Mobile support remains to be declared from testing, not assumed.

Authoritative references (checked 1 October 2026)

MDN: `showDirectoryPicker` - limited availability, secure context, user activation.

<https://developer.mozilla.org/en-US/docs/Web/API/Window/showDirectoryPicker>

MDN: `webkitdirectory` and `File.webkitRelativePath` - directory selection and relative hierarchy.

<https://developer.mozilla.org/en-US/docs/Web/API/HTMLInputElement/webkitdirectory>

<https://developer.mozilla.org/en-US/docs/Web/API/File/webkitRelativePath>

Vue TypeScript tooling: <https://vuejs.org/guide/typescript/overview>

Vite guide: <https://vite.dev/guide/>

The live GitHub roadmap and issue acceptance criteria remain authoritative for current progress.